



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Proyecto final:

Implementacion en Raspberry pi pico 2W del juego de
Galaga inalambrico.

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Microcomputadoras

Grupo: 03 - Semestre: 2026-2

Calf: _____



1. Objetivo:

Diseñar e implementar un videojuego estilo Galaga completamente funcional sobre una Raspberry Pi Pico 2W programada en MicroPython, capaz de ejecutarse en tiempo real sobre una pantalla TFT ILI9341 de 320x240 pixeles, siendo controlado de forma inalámbrica mediante un segundo microcontrolador Pico 2W que actúa como mando Bluetooth Low Energy (BLE) equipado con una unidad de medición inercial (IMU) MPU-6050 y dos botones físicos uno para disparar y otro para poner pausa. El sistema combina varios componentes de hardware y software para aprovechar al máximo las capacidades del microcontrolador. Utiliza comunicación SPI para mostrar gráficos rápidamente en la pantalla, protocolo I2C para leer datos del sensor inercial, Bluetooth Low Energy para la conexión inalámbrica, y PWM para generar sonidos con un buzzer. Además, emplea programación asíncrona con `uasyncio` para que todas las funciones del juego trabajen al mismo tiempo de manera organizada y eficiente.

2. Desarrollo:

2.1. Descripción General del Sistema

El proyecto se compone de dos unidades físicas independientes que se comunican de forma inalámbrica: la Consola, que contiene la pantalla TFT y el buzzer, y el Mando, que integra el sensor IMU y los botones de control. La arquitectura de software sigue un modelo cliente-servidor BLE donde el Mando actúa como periférico (servidor GATT) y la Consola actúa como central (cliente GATT).

Tabla 1: Componentes de hardware del sistema

Módulo	Componente	Interfaz	Función
Consola	Raspberry Pi Pico 2W	—	CPU principal del juego
Consola	TFT ILI9341 320x240	SPI0 (40 MHz)	Pantalla de juego
Consola	Buzzer Piezoeléctrico	PWM (GP15)	Efectos de sonido
Mando	Raspberry Pi Pico 2W	—	CPU del controlador
Mando	MPU-6050 (IMU)	I2C0 (400 kHz)	Control de movimiento
Mando	Botón Disparo (GP4)	GPIO Pull-up	Disparar proyectil
Mando	Botón Pausa (GP5)	GPIO Pull-up	Pausar / menú



2.2. Arquitectura de Software

El software de la Consola se organiza en módulos independientes que encapsulan la lógica de cada entidad del juego. El punto de entrada `main.py` lanza dos tareas asíncronas concurrentes mediante `uasyncio`: `ble_central_task()` para mantener la conexión con el mando, y `game_task()` para ejecutar el bucle principal del juego. Esta arquitectura garantiza que la pérdida temporal de conexión BLE no congele la lógica de juego.

Tabla 2: Módulos de software de la Consola

Módulo	Responsabilidad
<code>main.py</code>	Inicialización de hardware (SPI, display, buzzer), lanzamiento de tareas <i>async</i> BLE y juego.
<code>game.py</code>	Clase Game : bucle principal, lógica de juego, detección de colisiones, renderizado, gestión de niveles y menús.
<code>player.py</code>	Clase Player : posición, movimiento analógico basado en IMU, dibujo del sprite y animación de explosión.
<code>enemy.py</code>	Clase Enemy : posición en formación, animación alternada de sprites, puntuación por fila y explosión.
<code>asteroid.py</code>	Clases Asteroid , Guardian y DiveBomber : enemigos especiales con comportamientos de movimiento propios.
<code>bullet.py</code>	Clase Bullet : proyectiles del jugador y enemigos con velocidad, color y límites de pantalla configurables.
<code>buzzer.py</code>	Clase Buzzer : cola de tonos PWM no bloqueante con melodías y efectos de sonido para cada evento.
<code>virtual_button.py</code>	Clase VirtualButton : desacopla la lógica de pulsación (<i>pressed</i> / <i>held</i>) del estado BLE recibido.
<code>colours.py</code>	Constantes de colores en formato RGB565 y paleta de colores por nivel.

2.2.1. Módulo `main.py`: Orquestación y Conectividad

El archivo principal coordina la inicialización de periféricos (I2C, SPI) y administra el ciclo de vida de la aplicación mediante programación asíncrona (`uasyncio`).



2.2.2. Inicialización de Periféricos y Sensores

El Mando requiere inicializar el bus I2C para comunicarse con la Unidad de Medición Inercial (IMU) MPU-6050.

```
1 i2c = I2C(0, sda=Pin(8), scl=Pin(9), freq=400000)
2 imu = MPU6050(i2c)
3
4 btn_fire = Pin(4, Pin.IN, Pin.PULL_UP)
5 btn_pause = Pin(5, Pin.IN, Pin.PULL_UP)
```

Listing 1: Configuración del bus I2C y pines de entrada

La frecuencia de 400 kHz (Fast Mode) asegura que la lectura de los acelerómetros no se convierta en un cuello de botella. Los botones utilizan resistencias *Pull-Up* internas, evitando componentes externos adicionales.

2.2.3. Empaquetado de Datos Bluetooth (BLE)

Para transmitir la telemetría del mando a la consola, se empaquetan los datos en una estructura binaria ligera.

```
1 ax = imu.accel.x
2 ay = imu.accel.y
3
4 fire_pressed = (btn_fire.value() == 0)
5 pause_pressed = (btn_pause.value() == 0)
6
7 payload = struct.pack('<ffbb', ax, ay, int(fire_pressed), int(
    pause_pressed))
8 _char.write(payload)
```

Listing 2: Estructuración y transmisión de la trama de control

La función `struct.pack('<ffbb', ...)` codifica dos valores de punto flotante de 32 bits (aceleración X e Y) y dos bytes enteros (estado de los botones) en formato *Little-Endian*. Esta trama de 10 bytes minimiza el ancho de banda consumido por el enlace BLE.



2.3. Módulo `game.py`: Motor Principal del Videojuego

Con más de 600 líneas, este script concentra la máquina de estados, renderizado SPI y detección matemática de colisiones.

2.4. Renderizado Diferencial (Optimización SPI)

Para evitar parpadeos (*flickering*) y saturación del bus SPI, el juego solo redibuja los píxeles estrictamente necesarios.

```
1 def _render(self):
2     fh = self.display.fill_hrect
3     # Diferencia horizontal
4     dx = self.player.x - self.player._px
5     if dx > 0:
6         fh(self.player._px, self.player._py, dx, PLAYER_H, BLACK)
7     elif dx < 0:
8         fh(self.player.x + PLAYER_W, self.player._py, -dx, PLAYER_H,
9         BLACK)
10
11     # Dibujar la nueva posicion
12     self.player.draw(self.display)
```

Listing 3: Algoritmo de renderizado por deltas de posición

En lugar de limpiar toda la pantalla, el motor calcula la diferencia matemática (dx) entre el fotograma actual y el anterior. Luego, rellena con un rectángulo negro (`fill_hrect`) únicamente la estela dejada por el movimiento del sprite.

2.5. Efecto Parallax Dinámico (Estrellas)

El fondo se genera algorítmicamente sin cargar imágenes estáticas.

```
1 def _update_and_draw_stars(self):
2     dp = self.display.draw_pixel
3     for s in self.stars:
4         dp(s[0], s[1], BLACK) # Borrar anterior
5         s[1] += s[2]          # Desplazar verticalmente
6
7     # Efecto aleatorio de titileo usando operaciones de bits
```



```
8         if getrandbits(6) == 0:
9             flicker = (getrandbits(3) % 7) - 3
10            b = s[3] + flicker
11            s[4] = self._gray565(max(0, min(b * 8, 31)))
12            dp(s[0], s[1], s[4])
```

Listing 4: Generación de estrellas y efecto de titileo (*Flicker*)

El arreglo de estrellas se procesa de forma vectorial. El uso de `getrandbits` agiliza el cálculo aleatorio.

2.6. Módulo `player.py`: Cinemática del Jugador

Administra la física de la nave controlada por el usuario.

2.6.1. Filtro de Ruido Mecánico (Zona Muerta)

```
1 def move_analog(self, force_x, force_y):
2     mov_x = int(force_x * 15)
3     mov_y = int(force_y * 15)
4
5     if abs(force_y) > 0.15: # Filtro de Zona Muerta
6         self.x -= mov_y
7         if self.x < 0:
8             self.x = 0
9         if self.x > self._width - PLAYER_W:
10            self.x = self._width - PLAYER_W
```

Listing 5: Conversión analógico-digital de los vectores de aceleración

El umbral de 0.15 G permite ignorar pequeñas vibraciones o movimientos involuntarios de la mano del usuario que podrían generar lecturas incorrectas en la IMU.

2.7. Módulo `asteroid.py`: IA Numérica y Trayectorias

Controla enemigos con patrones de vuelo no estandarizados.

2.7.1. Generación de Rutas Sinusoidales



```
1 def update(self, frame_count, interval, speed, enemies):
2     ...
3     self.y += speed
4     # Movimiento oscilatorio armónico acoplado al eje Y
5     self.x = self._width // 2 + int(20 * math.sin(3 * 2 * math.pi *
        self.y / self._height))
```

Listing 6: Vuelo paramétrico del DiveBomber basado en funciones trigonométricas

El enemigo especial no desciende en línea recta. Calcula su componente *x* evaluando la función `math.sin()` con base en su progreso vertical (*y*), generando un patrón ondulante difícil de evadir.

2.8. Módulo `enemy.py`: Animación y Gestión del Enjambre

Optimiza el consumo de memoria en la representación visual de decenas de alienígenas.

2.9. Alternancia de Texturas por Enmascaramiento de Bits

```
1 def draw(self, display, frame_count):
2     if self.alive:
3         pair = _ALIEN_PAIRS[self.row % 3]
4         sprite = pair[0] if frame_count & 0x8 else pair[1]
5         display.draw_sprite(sprite, self.x, self.y, ALIEN_W, ALIEN_H)
```

Listing 7: Lógica de cambio de *Sprite* usando operaciones lógicas a nivel de bits

En vez de utilizar variables booleanas de alternancia que requieren ciclos de RAM, la instrucción evalúa el octavo bit del contador de frames general (`frame_count & 0x8`). Cada 8 cuadros, el resultado de la compuerta AND cambia de 0 a 1, intercalando el sprite de forma natural y ultra eficiente.

2.10. Módulo `bullet.py`: Manejo Dinámico de proyectiles

Define las reglas de existencia y destrucción balística.

```
1 def update(self):
2     self.y += self.dy
3     # Desactivar si supera las fronteras del framebuffer
```



```
4     if self.y < 0 or self.y > self._height:
5         self.active = False
```

Listing 8: Gestión del ciclo de vida del proyectil

El proyectil se vuelve inactivo tan pronto como abandona la geometría de la pantalla. Esto indica al bucle en `game.py` que debe eliminar la instancia de la lista principal para liberar la asignación dinámica de memoria (Previene memory leaks).

2.11. Módulo `buzzer.py`: Subsistema Acústico (PWM)

Opera el actuador piezoeléctrico mediante Modulación por Ancho de Pulso de forma no bloqueante.

```
1 def _start_pwm(self, freq):
2     if self.pwm is not None:
3         self.pwm.deinit()
4     if freq > 0:
5         self.pwm = PWM(Pin(self.pin_num))
6         self.pwm.freq(freq)
7         self.pwm.duty_u16(512) # Ciclo de trabajo del 50%
```

Listing 9: Encolamiento asíncrono y control del temporizador de hardware

Al enviar `duty_u16(512)` a un PWM con resolución típica de 10 bits ($2^{10} = 1024$), se genera una onda cuadrada simétrica perfecta (50 % High, 50 % Low) maximizando el volumen de salida acústico.

2.12. Módulo `virtual_button.py`: Filtro Lógico de Entradas

Protección contra el rebote eléctrico de los microinterruptores.

```
1 def is_pressed(self):
2     pressed_event = self._is_pressed and not self._was_pressed
3     if pressed_event:
4         self._was_pressed = True
5     return pressed_event
```

Listing 10: Detección de flanco ascendente (Debounce por Software)



Garantiza que una pulsación sostenida del usuario solo dispare una única acción lógica de lectura evaluando el estado del fotograma previo (`_was_pressed`).

2.13. Módulo `colours.py`: Perfiles de Renderizado

Codificación cromática directa para bus SPI.

```
1 BLACK    = 0x0000
2 RED      = 0xF800
3 LIME     = 0x07E0
4 BLUE     = 0x001F
5 WHITE    = 0xFFFF
```

Listing 11: Declaración de paleta RGB565

El estándar ILI9341 lee colores de 16 bits (5 bits rojo, 6 bits verde, 5 bits azul). Predefinirlos como constantes hexadecimales ahorra la ejecución de la fórmula analítica en el microcontrolador.

2.14. Comunicación BLE

La comunicación entre el Mando y la Consola se implementa mediante Bluetooth Low Energy usando la biblioteca `aioble` de MicroPython. El Mando opera como periférico GATT y publica un servicio personalizado con UUID `12345678-1234-1234-1234-123456789abc`. Dentro de este servicio expone una característica de notificación con UUID complementario que transmite una estructura de 10 bytes empaquetados con `struct` a una tasa de 100 Hz (cada 10 ms).

La trama BLE contiene: dos valores `float32` en *little-endian* con las aceleraciones `ax` y `ay` del MPU-6050 (que corresponden a los ejes de inclinación del mando), y dos bytes con el estado booleano de los botones de disparo y pausa. La Consola, al recibir una notificación, deserializa la trama y actualiza un objeto `ControllerState` compartido que el bucle de juego lee en cada frame para actualizar el movimiento del jugador y los eventos de botón.

```
1 # Trama BLE: '<ffbb' -> ax (float), ay (float), fire (byte), pause (
    byte)
2 payload = struct.pack('<ffbb', ax, ay, int(fire_pressed), int(
    pause_pressed))
3 _char.write(payload)
```



```
4 await _char.notify(connection)
```

2.15. Motor de Juego

2.15.1. Bucle Principal y Renderizado

El bucle de juego se ejecuta a aproximadamente 30 fps (*sleep* de 33 ms por iteración). En cada frame: se actualizan los estados de los botones virtuales, se aplica el movimiento analógico al jugador a partir de las aceleraciones del IMU, se procesan disparos, se actualizan todos los objetos activos, se detectan colisiones y finalmente se renderiza la escena. Para evitar parpadeo y minimizar la escritura SPI, el renderizado usa una estrategia de actualización diferencial: antes de cada frame se guardan las posiciones previas (*_save_positions*) y al dibujar solo se borra el rectángulo exacto del área anterior de cada sprite, pintándolo de negro antes de redibujar en la nueva posición.

2.15.2. Fondo Estrellado

El fondo del juego simula un campo de 30 estrellas con velocidades de desplazamiento vertical distintas (1, 2 o 3 píxeles por frame) y brillo variable, creando un efecto de paralaje y destello. Cada estrella almacena su posición, velocidad, brillo base y color en formato RGB565. Con una probabilidad de 1/64 por frame, el brillo de una estrella fluctúa aleatoriamente en ± 3 unidades para simular el centelleo. Al salir de la pantalla por la parte inferior, la estrella se reinicia en la parte superior con una posición horizontal aleatoria.

2.15.3. Enemigos y Formación

La formación inicial consta de 24 enemigos distribuidos en 3 filas de 8 columnas, con separación de 24 píxeles horizontales y 20 píxeles verticales. El movimiento de la formación es por filas rotativas: en cada tick de movimiento se desplaza una fila, avanzando a la siguiente en el ciclo. Cuando cualquier enemigo vivo de la fila activa alcanza el borde de la pantalla, toda la formación desciende *ENEMY_DROP* (10 px) y el sentido horizontal se invierte. La velocidad del tick de movimiento es inversamente proporcional al número de enemigos vivos, acelerando la formación conforme se eliminan enemigos.



2.15.4. Enemigos Especiales

El juego incluye tres tipos de enemigos especiales que aparecen periódicamente:

- **Asteroid** (bola de fuego): cae en línea recta con leve deriva horizontal. Destruye enemigos de la formación al impactarlos. Al colisionar con el jugador le quita una vida.
- **Guardian**: cae con movimiento descendente y deriva horizontal. Su efecto especial al colisionar con el jugador es sumar una vida (hasta el máximo de 3) en lugar de restarla, funcionando como power-up.
- **DiveBomber**: se lanza desde la posición de un enemigo vivo de la fila 2 y sigue una trayectoria sinusoidal hacia abajo usando la función `math.sin`. Vale 50 puntos al ser destruido.

2.15.5. Sistema de Disparos

El jugador puede disparar de dos formas: pulsación rápida (`is_pressed`), que genera un proyectil por evento, y disparo automático sostenido (`is_held`), que genera un proyectil cada 3 frames mientras el botón permanece presionado. Los proyectiles del jugador viajan hacia arriba a 5 píxeles por frame en color cyan, y los proyectiles enemigos viajan hacia abajo a 4 píxeles por frame en color naranja. La cadencia de disparo enemigo está controlada por `enemy_rate_of_fire`, que disminuye con cada nivel para incrementar la dificultad.

2.15.6. Detección de Colisiones

Las colisiones se calculan por AABB (*Axis-Aligned Bounding Box*) en cada frame sobre los siguientes pares de entidades: proyectiles del jugador contra enemigos de formación, proyectiles del jugador contra Guardian y DiveBomber, proyectiles enemigos contra el jugador, Asteroid y DiveBomber contra el jugador, Guardian contra el jugador (power-up), y Asteroid contra enemigos de formación. Una colisión jugador-enemigo directo también es posible si la formación desciende hasta la posición del jugador.

2.16. Sistema de Audio

El módulo Buzzer implementa una cola de reproducción no bloqueante para el buzzer piezoeléctrico conectado al pin GP15. En lugar de usar delays bloqueantes, el método `update()` es invocado en cada iteración del bucle de juego y del bucle del menú. Si hay una nota activa



y su duración ha transcurrido, detiene el PWM e inicia la siguiente nota de la cola. Las melodías están predefinidas como secuencias de frecuencias y duraciones, cubriendo eventos como: disparo del jugador, explosión, disparo enemigo, impacto, recogida de power-up, muerte del jugador, cambio de nivel, inicio de partida y notas del menú.

2.17. Dificultad Progresiva

El juego implementa un sistema de 10 niveles de dificultad que se aplican cíclicamente (nivel 11 usa la configuración del nivel 1, etc.). Además, el jugador puede seleccionar uno de tres modos de dificultad desde el menú de pausa:

- **Fácil:** velocidad de enemigos reducida al 60 %, intervalos de aparición y cadencia de disparo aumentados 50 %.
- **Normal:** parámetros base definidos por la tabla de niveles.
- **Difícil:** velocidad de enemigos aumentada 50 %, intervalos reducidos al 60 %, velocidades de asteroides y dive bombers incrementadas 50 %.

Tabla 3: Parámetros de dificultad por nivel (modo Normal)

Nivel	Vel. Enemigo (frames)	Intervalo Asteroide	Vel. Asteroide	Int. DiveBomber	Vel. DiveBomber
1	60	600	2	500	2
2	50	400	3	480	3
3	40	300	4	460	4
4	30	200	5	440	5
5	20	100	6	260	5
6	10	80	7	240	6
7	5	60	8	220	7
8	3	60	9	60	8
9	3	60	10	60	8
10	3	60	11	60	9

2.18. Menús y Pantallas

El flujo de pantallas del juego sigue la siguiente secuencia:

- **Pantalla de inicio (Splash Screen):** muestra el nombre de la institución, la materia y los integrantes del equipo. Espera a que el jugador presione el botón de disparo para continuar.



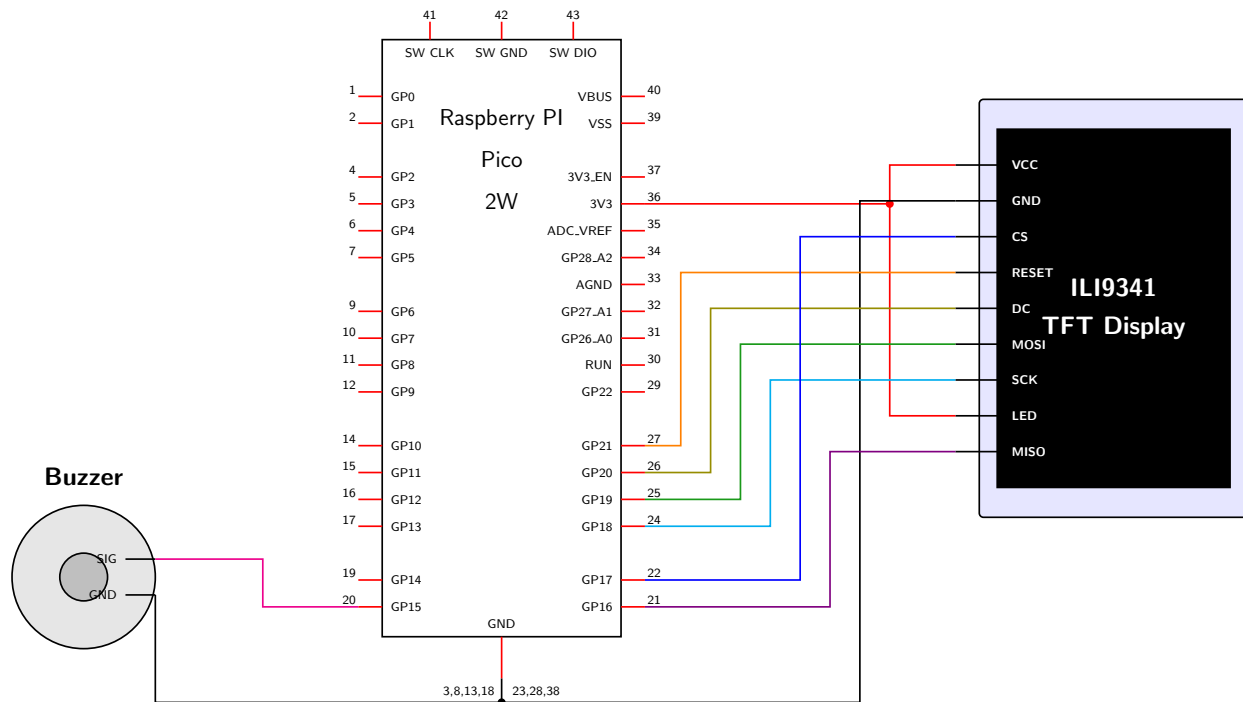
- **Pantalla de información:** muestra los sprites de todos los tipos de enemigos con su valor en puntos. El Guardian aparece con el indicador '+1 vida'.
- **Bucle de juego:** la partida en sí. El botón de pausa abre el menú de pausa en cualquier momento.
- **Menú de pausa:** permite continuar la partida, cambiar el modo de dificultad (cíclicamente entre Fácil, Normal y Difícil) o salir al menú principal. La navegación usa la inclinación del mando sobre el eje X.
- **Pantalla de Game Over:** muestra el puntaje final y el nivel alcanzado. Espera a que el jugador presione disparo para reiniciar.

2.19. Control Analógico por IMU

El movimiento del jugador se controla inclinando el mando físico. El MPU-6050 mide la aceleración en los ejes X e Y, que corresponden a la inclinación lateral y frontal del mando respectivamente. Estos valores se transmiten como `floats` por BLE y se aplican en el método `move_analog()` del jugador, donde se multiplican por un factor de 15 píxeles para obtener el desplazamiento por frame. Se aplica una zona muerta del 15 % ($|fuerza| > 0.15$) para evitar deriva con el mando en reposo. El movimiento sobre el eje Y de la Pico controla la posición horizontal en pantalla, y el eje X controla la profundidad vertical (dentro de la mitad inferior de la pantalla).

A continuación se presentan los diagramas de conexión de los dos módulos del sistema. Cada diagrama muestra el cableado completo entre la Raspberry Pi Pico 2W y sus periféricos, incluyendo los pines de alimentación, tierra y señales de datos. Las conexiones siguen un código de color para facilitar su identificación: rojo para 3.3 V, negro para GND, y colores distintos para cada señal de datos.

3. Diagrama de conexiones del modulo principal



La consola concentra la pantalla TFT ILI9341 y el buzzer piezoeléctrico. La pantalla se conecta al bus SPI0 de la Pico 2W usando cuatro pines de datos (MOSI, MISO, SCK, CS) más dos pines de control (DC y RST). El buzzer se controla mediante la salida PWM del pin GP15. Tanto la pantalla como el buzzer comparten el plano de tierra de la Pico, y la pantalla recibe su alimentación y la iluminación del *backlight* desde el pin 3V3.

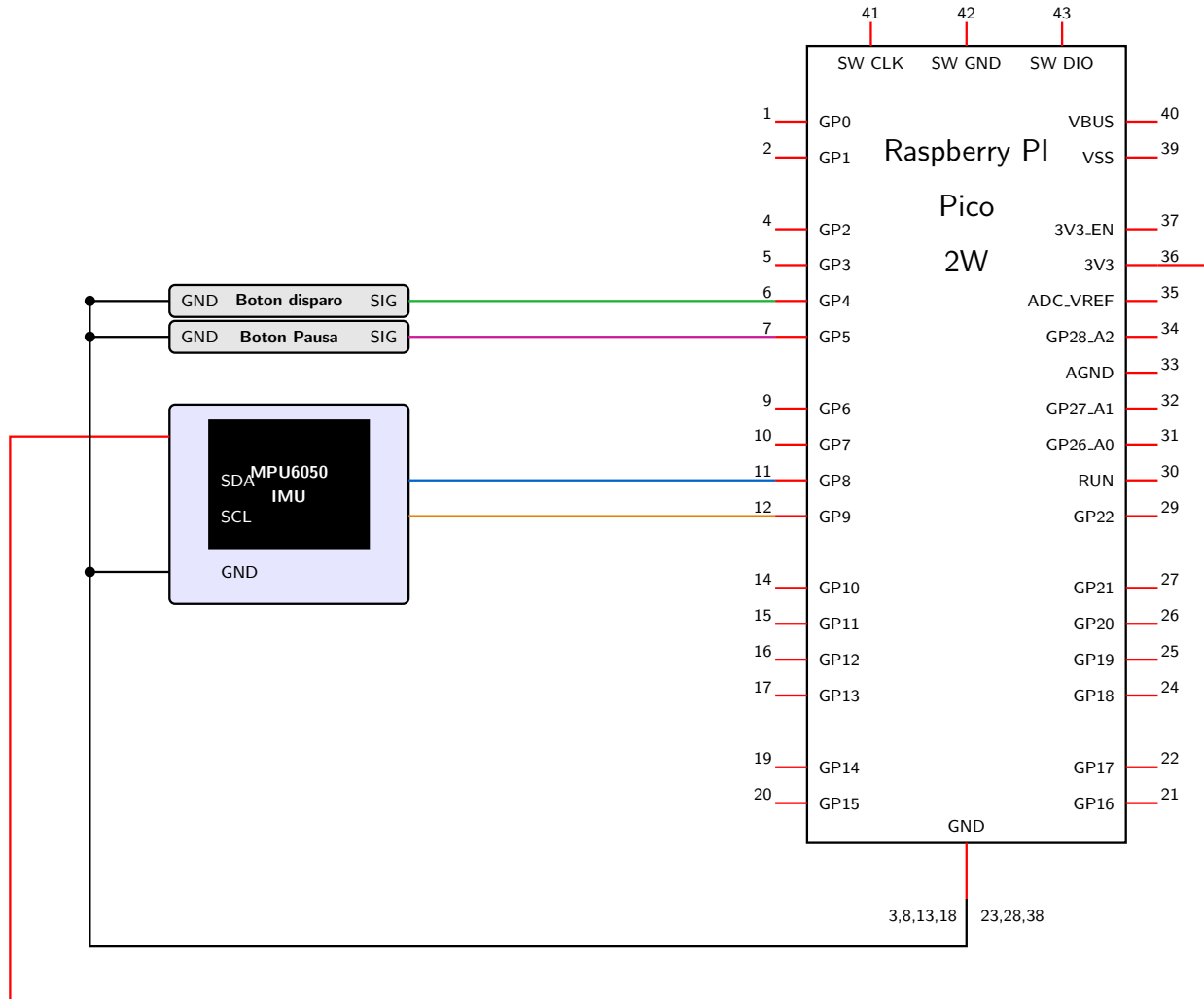


Tabla 4: Conexiones del módulo principal (Consola)

Periférico	Pin Periférico	Pin Pico 2W	Descripción
TFT ILI9341	VCC	3V3 (Pin 36)	Alimentación 3.3 V
TFT ILI9341	GND	GND (Pin 38)	Tierra
TFT ILI9341	CS	GP17 (Pin 22)	Chip Select SPI
TFT ILI9341	RESET	GP21 (Pin 27)	Reset de display
TFT ILI9341	DC	GP20 (Pin 26)	Dato / Comando
TFT ILI9341	MOSI	GP19 (Pin 25)	Datos SPI (salida)
TFT ILI9341	SCK	GP18 (Pin 24)	Reloj SPI (40 MHz)
TFT ILI9341	LED	3V3 (Pin 36)	Backlight (compartido)
TFT ILI9341	MISO	GP16 (Pin 21)	Datos SPI (entrada)
Buzzer	SIG	GP15 (Pin 20)	Señal PWM de audio
Buzzer	GND	GND (Pin 38)	Tierra

El bus SPI se configura a 40 MHz de frecuencia de reloj con polaridad y fase en 0 (modo SPI 0). La clase `Display` del driver ILI9341 maneja internamente los comandos de inicialización de la pantalla y expone los métodos `draw_sprite()`, `fill_hrect()`, `draw_pixel()` y `draw_text8x8()` que utiliza el motor de juego.

4. Diagrama de conexiones del mando



El mando integra el sensor IMU MPU-6050 para la detección de movimiento por inclinación y dos botones físicos con resistencias de *pull-up* internas habilitadas por software. El MPU-6050 se conecta al bus I2C0 de la Pico utilizando los pines GP8 (SDA) y GP9 (SCL) a una frecuencia de 400 kHz. Los botones se conectan directamente entre el pin GPIO y GND, aprovechando las resistencias de *pull-up* internas del RP2350. La alimentación del MPU-6050 se toma del pin 3V3 de la Pico y se enruta externamente por debajo del circuito para evitar cruces con las señales de datos.

Tabla 5: Conexiones del módulo del mando

Periférico	Pin Periférico	Pin Pico 2W	Descripción
MPU-6050	VCC	3V3 (Pin 36)	Alimentación 3.3 V
MPU-6050	GND	GND (Pin 38)	Tierra
MPU-6050	SDA	GP8 (Pin 11)	Dato I2C (400 kHz)
MPU-6050	SCL	GP9 (Pin 12)	Reloj I2C (400 kHz)
Botón Disparo	SIG	GP4 (Pin 6)	Entrada con Pull-up interno
Botón Disparo	GND	GND (común)	Tierra
Botón Pausa	SIG	GP5 (Pin 7)	Entrada con Pull-up interno
Botón Pausa	GND	GND (común)	Tierra

El software del mando (`main.py` del mando) lee las aceleraciones del MPU-6050 utilizando la librería `imu` (clase `MPU6050` sobre I2C), muestrea el estado de los botones en cada iteración del bucle asíncrono y construye la trama BLE de 10 bytes que se transmite vía notificación GATT cada 10 ms. La gestión de memoria se optimiza con llamadas periódicas a `gc.collect()` para evitar fragmentación en el heap de MicroPython durante la ejecución continua.

4.1. Protocolo BLE

Ambos microcontroladores utilizan el módulo `aioble` sobre el stack Bluetooth integrado del chip RP2350 de la Pico 2W. El servicio GATT se define con UUIDs de 128 bits personalizados para evitar conflictos con servicios estándar. La característica de datos se configura con permisos de lectura y notificación en el periférico. La Consola realiza un escaneo activo de 5 segundos buscando el nombre de dispositivo `'PicoMando'`; al encontrarlo, establece la conexión, descubre el servicio y se suscribe a las notificaciones. Ante cualquier desconexión o error, el task BLE espera 2 segundos y reintenta automáticamente, garantizando la reconexión sin intervenir la lógica de juego.

```
1 _SERVICE_UUID = bluetooth.UUID('12345678-1234-1234-1234-123456789abc')
2 _CHAR_UUID     = bluetooth.UUID('87654321-4321-4321-4321-cba987654321')
```



5. Resultados:



6. Conclusiones:

- Espinoza Matamoros Percival Ulises:
- Flores Colin Victor Jaziel:
- García Cortés Adolfo de Jesús:
- Lara Hernandez Angel Husiel:
- Lugo Manzano Rodrigo: